Auditing Closed-Loop Learning in Recurrent Neural Networks: Reproduction, Robustness, and Generalization

Anonymous authors
Paper under double-blind review

Abstract

Recurrent neural networks are often used as mechanistic models of learning and control, but closed-loop training creates reproducibility challenges because the model's actions alter future inputs. We conduct a claim-level reproducibility study of Ger and Barak's analysis of closed-loop RNN learning dynamics. Rather than asking only whether original figures can be regenerated, we test whether the paper's main claims survive independent implementation, seed variation, optimizer and hyperparameter perturbations, coupled-system stability diagnostics, multi-horizon tradeoff tests, and transfer to nonlinear or path-integration-style settings. Across 455 completed paired runs, the central open-loop/closed-loop divergence reproduces strongly: under the original double-integrator setting, 50/50 paired seeds show lower deployed loss for closed-loop training, with mean final deployed losses 0.0933 for closed-loop training versus 0.3119 for open-loop training. The coupled-system stability transition also reproduces in 50/50 original seeds, and a targeted control-penalty sweep quantifies short-vs-long horizon tradeoffs in 120/120 runs. However, the stage claim is more nuanced: all original seeds show an early rapid-improvement phase followed by slow progress, but an objective three-stage changepoint criterion finds no robust second boundary. Generalization is partial: 42/90 architecture/task extension runs preserve deployed-loss divergence, with strong transfer across tanh/GRU/low-rank variants but weak transfer to ring path-integration tasks. These results motivate concrete reporting practices for closed-loop RNN studies: deployed closed-loop loss, paired-seed comparisons, algorithmic stage criteria, coupled-system spectra, feedback strength, rollout horizon, and seed-level failure rates should be reported whenever papers make claims about recurrent learning dynamics in interactive environments.

1 Introduction

Closed-loop recurrent neural networks occupy an important middle ground between supervised sequence modeling, control, and computational neuroscience. In a closed-loop environment, an RNN's output changes the future inputs it receives. This feedback makes the training problem qualitatively different from open-loop imitation or teacher-forced supervised learning, where future inputs are independent of the learner's deployed actions. Ger and Barak (Ger and Barak, 2025) recently argued that this distinction produces learning dynamics that are fundamentally different from those of otherwise identical open-loop RNNs. Their paper is valuable because it makes a mechanistic claim rather than only a performance claim: the coupled agent-environment system, not the RNN alone, determines important learning transitions.

This paper is not a figure-level reproduction study. Instead, we use Ger and Barak's closed-loop RNN learning framework as a test case for claim-level reproducibility in recurrent control systems. We ask which conclusions survive independent implementation, seed variation, optimizer and hyperparameter perturbations, coupled-system stability analysis, and transfer to nonlinear or path-integration-style settings. This framing matters for TMLR-style reproducibility: a rerun of a public artifact can be educational, but it is only broadly useful

when it surfaces reusable lessons about when claims are reliable, when they are protocol-sensitive, and what future papers should report.

Our study makes four contributions. First, we provide an independent, config-driven reproduction of the central open-loop/closed-loop learning comparisons in Ger and Barak. Second, we convert qualitative claims about learning stages and stability transitions into quantitative, seed-level tests with confidence intervals. Third, we measure robustness under perturbations to optimizer, initialization, feedback strength, noise, and rollout horizon, identifying which conclusions are stable and which are protocol-sensitive. Fourth, we distill an audit checklist for future closed-loop RNN studies, including paired-seed comparisons, deployed closed-loop loss, stage criteria, coupled-system stability diagnostics, and failure-rate reporting.

2 Related Work

Task-trained RNNs in computational neuroscience. Task-trained RNNs are widely used as mechanistic models because they can solve cognitive and motor tasks while exposing internal dynamics for analysis. Early and influential work showed that high-dimensional trained RNNs can implement low-dimensional dynamical mechanisms (Sussillo and Barak, 2013), and that recurrent dynamics can account for context-dependent computation in prefrontal cortex (Mante et al., 2013). This line of work has become a general modeling strategy in systems neuroscience, where RNNs serve as hypothesis-generating models rather than only predictive tools (Barak, 2017). At the same time, large-scale analyses of trained recurrent networks show that architecture and training details can change representational geometry and dynamics even when task performance is similar (Maheswaranathan et al., 2019). This motivates our claim-level framing: reproducing a task curve is not enough if the mechanistic conclusion depends on training protocol or model class.

Dynamical-systems analysis of RNNs. The original paper sits in a tradition of reverse-engineering trained RNNs through fixed points, low-dimensional projections, eigenvalues, and linearized dynamics. Sussillo and Barak's low-dimensional analysis made the case that trained recurrent networks can be understood through dynamical-systems tools (Sussillo and Barak, 2013), and later work used related analyses to compare recurrent dynamics across large populations of networks (Maheswaranathan et al., 2019). Our audit differs in emphasis. We do not only ask whether a trained RNN has interpretable internal dynamics; we ask whether the relevant dynamical object is the RNN alone or the coupled agent-environment system. This distinction is central in closed-loop settings because the environment transforms actions into future observations.

Open-loop training, closed-loop deployment, and distribution shift. The mismatch between teacher-forced training and deployed rollouts is well known in sequence prediction and imitation learning. Scheduled sampling was proposed to reduce the discrepancy between training on ground-truth previous tokens and testing on model-generated tokens (Bengio et al., 2015). In imitation learning, DAgger and related reductions explicitly address the fact that a learner's actions change the future states it visits, invalidating i.i.d. supervised-learning assumptions (Ross et al., 2011). Closed-loop RNN learning has an analogous but mechanistically richer mismatch: open-loop imitation can minimize teacher-forced action error while still failing under deployed closed-loop rollouts. This connection motivates our insistence on reporting deployed closed-loop loss, not only open-loop or imitation loss.

Reproducibility and robustness in machine learning. Our audit follows the broader shift from artifact reruns to structured reproducibility studies. The NeurIPS reproducibility program emphasized clear artifacts, documented environments, and reproducibility checklists (Pineau et al., 2021). In deep reinforcement learning, Henderson et al. showed that algorithmic conclusions can be sensitive to seeds, evaluation protocol, and implementation details (Henderson et al., 2018). Closed-loop RNN studies share many of these risks because training outcomes can depend on random initialization, rollout horizon, optimizer, and environment coupling. The present paper therefore treats Ger and Barak's work as a case study for a reusable audit protocol: paired seeds, stress tests, coupled-system diagnostics, and reporting lessons.

3 Original Claims and Audit Questions

We separate the original claims under reproduction from audit questions introduced by this study. The reproduced claims are C1–C3. C1 is the central divergence claim: identical RNNs trained open-loop and closed-loop follow different learning trajectories and differ under closed-loop deployment. C2 is the stage claim: closed-loop learning passes through distinct learning stages. C3 is the mechanistic stability claim: progress depends on stability of the coupled agent-environment system.

The audit questions are A1–A2. A1 asks whether C1–C3 are robust to protocol perturbations and whether a targeted short-vs-long horizon analysis reveals the expected stability tradeoff. A2 asks whether the phenomena generalize across architectures and tasks. We use A1 and A2 to avoid implying that Ger and Barak made every robustness or generalization claim in exactly our framing.

For each reproduced claim or audit question, we define three evidence levels. A result is *reproduced* when the effect and diagnostic hold under the relevant protocol for most paired seeds, with confidence intervals excluding zero or a pre-specified smallest effect size of interest. For C1 this smallest effect is a 5 percent relative deployed-loss gap. A result is *partially reproduced* when the direction appears but is unstable across seeds, sensitive to plausible perturbations, or only weakly supported by the diagnostic. A result is *not reproduced* when the effect is absent, reverses, or depends on hand-picked visual criteria.

4 Closed-Loop RNN Reproducibility Audit Protocol

We specify a modest audit protocol rather than proposing a new benchmark. The protocol is a recipe for testing claims about closed-loop RNN learning dynamics in any recurrent control setting where actions can affect future observations. Its inputs are: a set of original claims, a task/environment, an open-loop training procedure, a closed-loop training procedure, a seed set, a small set of protocol perturbations, and at least one held-out architecture or task variant. Its outputs are: per-seed deployed closed-loop metrics, stage labels, stability diagnostics, perturbation summaries, generalization summaries, and a claim-level reproducibility matrix. Table 1 gives the concrete specification used in this paper.

The protocol deliberately separates *measurement* from *interpretation*. Measurement consists of paired runs, deployed losses, spectra, stage labels, and perturbation outcomes. Interpretation happens only after aggregating seed-level outputs into claim-level decisions. This separation is important because closed-loop RNN papers often make qualitative statements about stages or mechanisms from a small number of curves; our protocol requires those statements to be tied to explicit detectors, uncertainty estimates, and failure rates.

4.1 Exact metric definitions

Let $s_t \in \mathbb{R}^{d_s}$ be the environment state, $o_t = Cs_t + \epsilon_t$ the observation, h_t the RNN hidden state, and $u_t = f_\theta(o_t, h_t)$ the action. Closed-loop deployment follows

$$s_{t+1} = F(s_t, u_t), \qquad h_{t+1} = G_\theta(o_t, h_t). \tag{1}$$

For an episode of horizon T, the deployed closed-loop loss for model θ is

$$L_{\text{deploy}}(\theta) = \frac{1}{T}\sum_{t=0}^{T-1}\left[\ell_s(s_t, s_t^\star) + \lambda_u \|u_t\|_2^2\right], \tag{2}$$

where ℓ_s is squared state or tracking error and λ_u is the control penalty. For the double-integrator stabilization task, $\ell_s(s_t, 0) = \|s_t\|_2^2$.

Open-loop imitation is evaluated separately from deployed performance. Given a teacher policy $\theta^\star$ and teacher-generated observations $o_t^\star$, the teacher-forced imitation loss is

$$L_{\text{TF}}(\theta; \theta^\star) = \frac{1}{T}\sum_{t=0}^{T-1}\|f_\theta(o_t^\star, h_t) - f_{\theta^\star}(o_t^\star, h_t^\star)\|_2^2. \tag{3}$$

Table 1: Closed-loop RNN reproducibility audit protocol. The protocol is intentionally lightweight: each row defines a required audit step, its minimum implementation, and the output needed to support claim-level conclusions.

Audit step	**Minimum requirement**	**Recorded output**
Paired training	Clone the same initialization into open-loop and closed-loop learners; keep architecture and evaluation budget fixed.	Per-seed train loss, deployed closed-loop test loss, peak deployed loss, final loss, and effective-gain trajectory for both learners.
Seed-level uncertainty	Run enough seeds to estimate direction agreement and confidence intervals; report seed count explicitly.	Mean, 95 percent bootstrap CI, fraction of seeds supporting each claim, and seed-level failure/recovery rates.
Stage detection	Predefine a detector using log-loss derivative, minimum plateau duration, and optional stability crossing.	Stage labels, plateau duration, plateau loss, transition epochs, and frequency of detected stages across seeds/settings.
Coupled diagnostics	Compute RNN-only and coupled agent-environment spectra whenever the system can be linearized or locally linearized.	Spectral radius time series, stability-crossing epoch, plateau-exit gap, and predictive correlation with recovery/failure.
Robustness perturbations	Change factors that alter optimization or environment coupling, such as optimizer, initialization scale, feedback strength, noise, horizon, control penalty, and clipping.	Claim support under each perturbation, effect-size changes, and perturbation-level failure rates.
Generalization check	Test at least one architecture or task variant that preserves action-dependent inputs and one that changes the recurrent/control structure.	Claim support by variant and a boundary-condition statement explaining when the phenomenon does or does not transfer.
Claim/audit decision	Classify each claim or audit question as reproduced, partially reproduced, or not reproduced using the same criteria across settings.	Matrix linking evidence type, original setting, robustness, generalization, and actionable lesson.

The distinction between L_{TF} and L_{deploy} is central: C1 is supported only if the training-regime comparison is evaluated under deployed closed-loop rollouts.

To summarize the controller in the double-integrator setting, we estimate an effective linear feedback gain K_t from rollout states and actions. For a collection of state-action pairs (x_i, u_i) at training epoch t, with matrices $X = [x_1^\top; \ldots; x_n^\top]$ and $U = [u_1^\top; \ldots; u_n^\top]$, the ridge-stabilized estimate is

$$\widehat{K}_t = (X^\top X + \alpha I)^{-1} X^\top U, \tag{4}$$

reported as a row vector for scalar control. The closed/open gain-trajectory distance is

$$D_K = \left(\sum_{t \in \mathcal{E}} \left\| \widehat{K}_t^{\mathrm{CL}} - \widehat{K}_t^{\mathrm{OL}} \right\|_F^2 \right)^{1/2}, \tag{5}$$

where $\mathcal{E}$ is the set of recorded epochs. In the current implementation every epoch is recorded; future larger sweeps can subsample $\mathcal{E}$ without changing the definition.

For linear RNN controllers and linear environments, we compute both RNN-only and coupled-system spectra. With

$$s_{t+1} = As_t + Bu_t, \qquad h_{t+1} = (1-\eta)h_t + \eta(W_{hh}h_t + W_{ih}Cs_{t+1}), \qquad u_t = W_{ho}h_t, \tag{6}$$

the coupled transition matrix over $z_t = [s_t^\top, h_t^\top]^\top$ is

$$P_\theta = \begin{bmatrix} A & BW_{ho} \\ \eta W_{ih}CA & (1-\eta)I + \eta W_{hh} + \eta W_{ih}CBW_{ho} \end{bmatrix}. \tag{7}$$

The two spectral diagnostics are

$$\rho_{\mathrm{RNN}}(\theta) = \max_i |\lambda_i(W_{hh})|, \qquad \rho_{\mathrm{coup}}(\theta) = \max_i |\lambda_i(P_\theta)|. \tag{8}$$

We define the stability-crossing epoch as

$$\tau_\rho = \min \{t : \rho_{\mathrm{coup}}(\theta_t) < 1 \text{ and } \rho_{\mathrm{coup}}(\theta_{t:t+m}) < 1\}, \tag{9}$$

where m is a minimum persistence window. The plateau-exit gap is $\Delta_\rho = \tau_{\mathrm{exit}} - \tau_\rho$.

Stage labels are computed from smoothed log deployed loss $y_t = \log(\max(L_{\mathrm{deploy},t}, \varepsilon))$. Let $\dot{y}_t$ be a finite-difference derivative. Stage 1 ends at the first epoch τ_1 such that

$$|\dot{y}_{t:t+m}| \le \delta_{\mathrm{plat}} \tag{10}$$

for at least m consecutive epochs. The plateau exits at the first later epoch τ_2 such that

$$\dot{y}_t < -\delta_{\mathrm{improve}}, \tag{11}$$

or at the stability crossing τ_ρ when no derivative-based exit is detected. The plateau length is $\tau_2 - \tau_1$, and C2 support is the fraction of seeds/settings with plateau length at least m.

For the A1 short-vs-long horizon tradeoff audit, we evaluate the same closed-loop controller at short and long rollout horizons. Let $L_e^{(T_s)}$ and $L_e^{(T_\ell)}$ be deployed losses measured at training evaluation index e for short horizon T_s and long horizon T_ℓ. A myopic-improvement event occurs when

$$\Delta \log L_e^{(T_s)} < -\delta_{\mathrm{short}}, \tag{12}$$

and a tradeoff event occurs when this short-horizon improvement coincides with either long-horizon worsening or increased coupled spectral radius:

$$\Delta \log L_e^{(T_s)} < -\delta_{\mathrm{short}} \quad \text{and} \quad \left[\Delta \log L_e^{(T_\ell)} > \delta_{\mathrm{long}} \text{ or } \Delta\rho_{\mathrm{coup},e} > 0\right]. \tag{13}$$

We report both the unconditional tradeoff fraction over evaluation intervals and the conditional fraction among myopic-improvement intervals. In the targeted sweep, $T_s = 10$, $T_\ell = 200$, and $\delta_{\mathrm{short}} = \delta_{\mathrm{long}} = 0.005$.

For a scalar metric M and paired seed i, the open/closed effect is

$$\Delta_i(M) = M_i^{\mathrm{OL}} - M_i^{\mathrm{CL}}. \tag{14}$$

We report the mean effect $\bar{\Delta}$, a nonparametric bootstrap 95 percent confidence interval, and the direction-agreement fraction

$$a(M) = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\Delta_i(M) > 0]. \tag{15}$$

For a claim c with binary seed-level support indicator $q_i(c)$, the claim-support rate is

$$S(c) = \frac{1}{N} \sum_{i=1}^{N} q_i(c). \tag{16}$$

Failure rates are defined analogously by replacing $q_i(c)$ with indicators for divergence, non-finite loss, missing plateau, or failure to recover by the final epoch.

4.2 Paired open-loop and closed-loop training

For each seed, one initialization is cloned into an open-loop and a closed-loop learner. The closed-loop model is trained by rolling out in the environment; the open-loop model imitates teacher trajectories. Both are evaluated under deployed closed-loop rollout.

4.3 Stage detection

The original paper describes closed-loop learning stages visually. We operationalize C2 as early rapid improvement, a plateau or slow-progress phase, and post-transition improvement, detected from log-loss derivatives, a minimum plateau duration, and optionally a coupled spectral-radius crossing.

4.4 Coupled-system stability diagnostics

For linearized settings, we compute the spectral radius of the RNN recurrent matrix and of the coupled agent-environment transition matrix. This targets C3 directly: if closed-loop dynamics are governed by the agent-environment loop, the coupled diagnostic should be more informative than an isolated RNN diagnostic.

4.5 Robustness and generalization

The A1 robustness sweep changes learning rate, initialization scale, feedback strength, episode length, observation noise, control penalty, optimizer, and gradient clipping. The A2 generalization sweep tests nonlinear RNNs, GRUs, low-rank RNNs, tracking control, and ring/path-integration-style tasks.

4.6 Original artifact, independent implementation, and deviations

We preserved the public artifact under `external/original_artifact/`. It contains the original notebooks for figure reproduction, nonlinear training, theory, and tracking, plus the pickle files used by the artifact. Our reproduction log records which files load, which notebooks depend on precomputed data, and which notebook runs are long or checkpoint-heavy.

The experiments reported here use an independent, config-driven implementation rather than direct notebook execution. The double-integrator task, paired open/closed training, effective gains, coupled spectra, stage detectors, robustness perturbations, and generalization tasks were reimplemented as package modules and command-line scripts. We matched the original double-integrator architecture and training profile where needed for C1–C3, then intentionally introduced new A1/A2 audits over seeds, optimizer, initialization, feedback, horizon, noise, clipping, architecture, and task. We did not contact the original authors; all choices were made from the paper, public artifact, and documented configs. Exact notebook reruns are therefore treated as artifact checks, while the paper's quantitative claims are evaluated through the independent implementation. Every table and figure can be regenerated from saved raw CSVs with `scripts/run_full_audit.sh`, `scripts/run_targeted_c2_c4_c5.sh`, and `python -m closed_loop_repro.plotting.make_all_figures -config configs/figures/tmlr.yaml`.

5 Result 1: Direct Reproduction of the Core Divergence

The central open-loop/closed-loop divergence reproduces cleanly under the original double-integrator setting (Figure 1). Across 50 paired seeds, the final deployed closed-loop loss of the closed-loop trained model is 0.0933 with bootstrap 95 percent CI $[0.0932, 0.0935]$. The open-loop trained student, evaluated under deployed closed-loop rollout, has final loss 0.3119 with CI $[0.3044, 0.3193]$. The paired deployed-loss gap is therefore 0.2186 with CI $[0.2110, 0.2260]$, and the direction of the effect is reproduced in 50/50 seeds. Effective feedback gains also diverge: the final closed/open gain distance is 0.5045 with CI $[0.4998, 0.5092]$.

The result is not just a visual overlay of curves. It shows that minimizing open-loop imitation loss can produce a controller whose deployed closed-loop loss remains substantially higher than the directly closed-loop trained controller. This supports C1 and the main reporting lesson of the audit: closed-loop RNN studies should report deployed closed-loop performance, not only teacher-forced or open-loop training loss.

6 Result 2: Robustness and Tradeoff Audit

The robustness sweep consists of 195 paired runs: 13 perturbation settings with 15 seeds per setting (Table 2 and Figure 3). C1 is robust across most perturbations. It is supported in 165/195 robustness runs

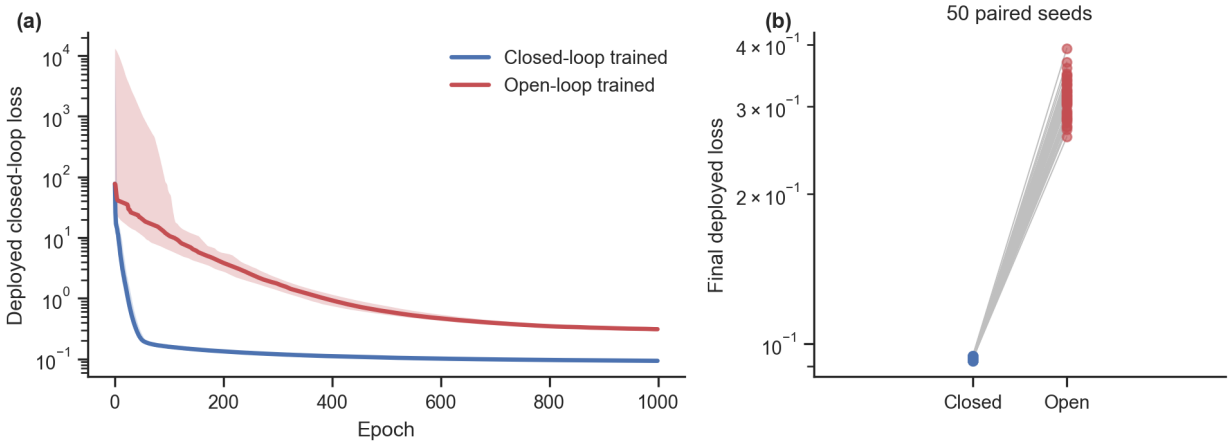

Figure 1: Core C1 reproduction under the original double-integrator setting. (a) Median deployed closed-loop test loss across paired seeds, with interquartile bands. (b) Paired final deployed losses for the same initializations. The direction of the deployed-loss gap reproduces in 50/50 seeds.

Table 2: Robustness perturbation grid. The original protocol also has the separate 50-seed core reproduction; the robustness grid repeats the baseline for 15 additional paired seeds so every perturbation has the same seed count.

Perturbation	**Changed value**	**Seeds**
`original`	Baseline double-integrator protocol	50 core; 15 grid
`lr_low`	learning rate 0.003	15
`lr_high`	learning rate 0.03	15
`init_weak`	recurrent initialization scale 0.03	15
`init_strong`	recurrent initialization scale 0.3	15
`feedback_low`	environment feedback strength 0.5	15
`feedback_high`	environment feedback strength 1.5	15
`episode_short`	rollout horizon 25 steps	15
`episode_long`	rollout horizon 100 steps	15
`obs_noise`	observation noise standard deviation 0.05	15
`adam`	optimizer Adam with learning rate 0.001	15
`high_control_penalty`	control penalty 0.05	15
`no_clip`	gradient clipping disabled	15

overall and in every finite run for the original, learning-rate, initialization, feedback-strength, episode-length, observation-noise, and high-control-penalty perturbations. The two important exceptions are informative. With Adam, the final deployed loss gap almost disappears: mean final closed-loop loss is 0.0780 and mean final open-loop deployed loss is 0.0798, yielding no C1 support under the 5 percent deployed-loss criterion. With gradient clipping disabled, all 15 runs in the `no_clip` condition produce non-finite final losses and are counted as failures rather than negative evidence for the mechanistic claim.

The stage claim is more sensitive to how "stage" is defined (Figure 2). A slow-progress phase is detected in 180/195 robustness runs, with failures concentrated entirely in the non-finite no-clipping condition. However, the stricter three-stage criterion, which requires both a plateau and a non-fallback second boundary, is supported in only 1/195 robustness runs. Thus the broad qualitative observation that closed-loop training can enter a slow-progress regime is robust, but the claim that three distinct stage boundaries are algorithmically stable is not supported by the current detector.

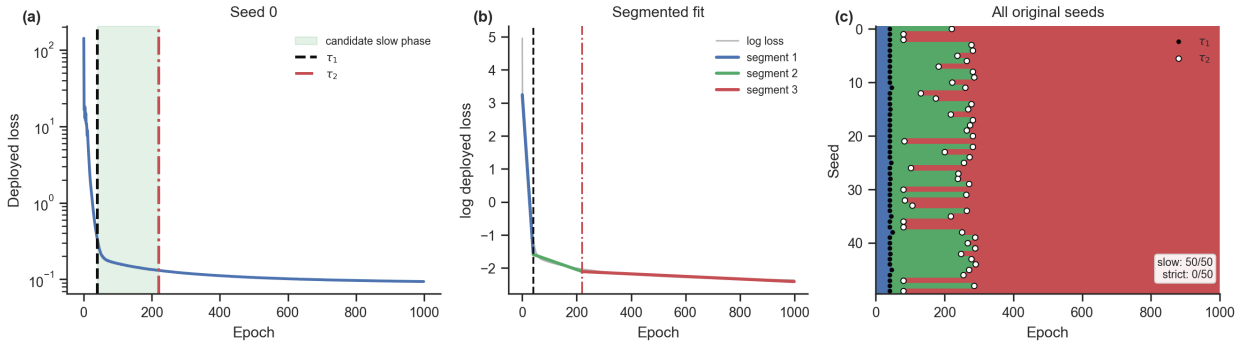

Figure 2: Stage analysis for the original-setting runs. (a,b) A representative seed with candidate boundaries τ_1 and τ_2 and the shaded candidate slow-progress phase. (c) Aggregate stage raster for all 50 original seeds; black dots mark τ_1 and white dots mark τ_2. The segmented detector uses minimum segment length 20 epochs, Stage 1 slope < -0.02, Stage 2 slope magnitude below $\max(0.35|\text{slope}_1|, 0.004)$, and Stage 3 reacceleration slope $< \text{slope}_2 - 0.002$. It finds rapid-improvement plus slow-progress evidence in 50/50 seeds but strict three-stage support in 0/50.

The coupled-system stability transition is robust wherever the coupled spectrum is available and the run remains finite. It is supported in 180/195 robustness runs, again failing only in the no-clipping condition. This supports the original mechanistic emphasis on the coupled agent-environment system and separates the stability claim from the more fragile stage-boundary claim.

The targeted A1 sweep supports a local short-vs-long horizon tradeoff (Figure 4). Across six control penalties and 20 seeds per penalty, the multi-horizon tradeoff criterion is supported in 120/120 runs. The mean unconditional tradeoff-event fraction is 0.0706 with bootstrap 95 percent CI $[0.0691, 0.0720]$, and the mean conditional tradeoff fraction among myopic-improvement intervals is 0.2814 with CI $[0.2744, 0.2881]$. The result is stable across control penalties from 0 to 0.1. Thus A1 is supported as an intermittent within-training tradeoff, not as a statement that final short-horizon loss and final long-horizon loss must move in opposite directions.

7 Result 3: Which Diagnostics are Predictive?

The original-setting coupled-system stability transition reproduces in 50/50 seeds (Figure 5). The final coupled spectral radius is finite and below the stability threshold in every original run, and the stability-crossing epoch occurs at the same point identified by the derivative-based plateau-start detector. This supports C3 in the original double-integrator setting.

However, the spectral diagnostic is architecture-dependent in the current implementation. For linear, tanh, and low-rank controllers on linear tasks, the coupled transition matrix can be constructed explicitly. We do not report coupled spectral diagnostics for GRU or ring variants because our predefined C3 diagnostic requires an explicitly constructed coupled transition matrix. Extending C3 to gated nonlinear controllers requires local Jacobian linearization, which we leave to future work.

The most important diagnostic distinction concerns C2. The coupled stability crossing is robust, but the strict three-stage plateau-exit boundary is not. In the original setting, a slow-progress phase is present in 50/50 seeds, yet the second boundary is a fallback in 50/50 seeds. A segmented-regression changepoint analysis likewise finds two log-loss segments that fit substantially better than one segment, but the third segment does not show the required post-plateau reacceleration. The median changepoint boundaries are epochs 40 and 255, with median slopes -0.1131, -0.00229, and -0.00036 for the three fitted segments. Thus our data support "rapid improvement followed by slow progress," but not a robust three-stage sequence with a clearly detected exit.

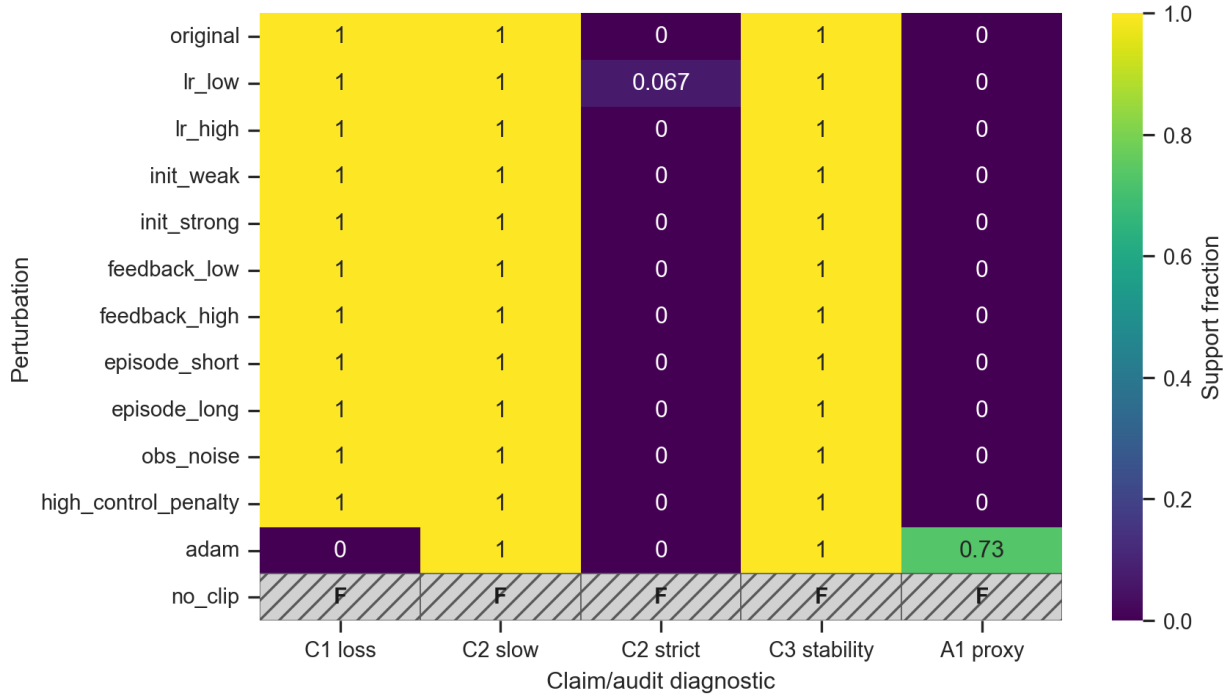

Figure 3: Robustness heatmap over perturbation settings and claim diagnostics. C1 and C3 are stable across most finite settings, C2 is robust only under the broad slow-progress criterion, the weak A1 proxy is not decisive outside the targeted multi-horizon sweep, and disabling gradient clipping produces non-finite failures marked as F rather than zero support.

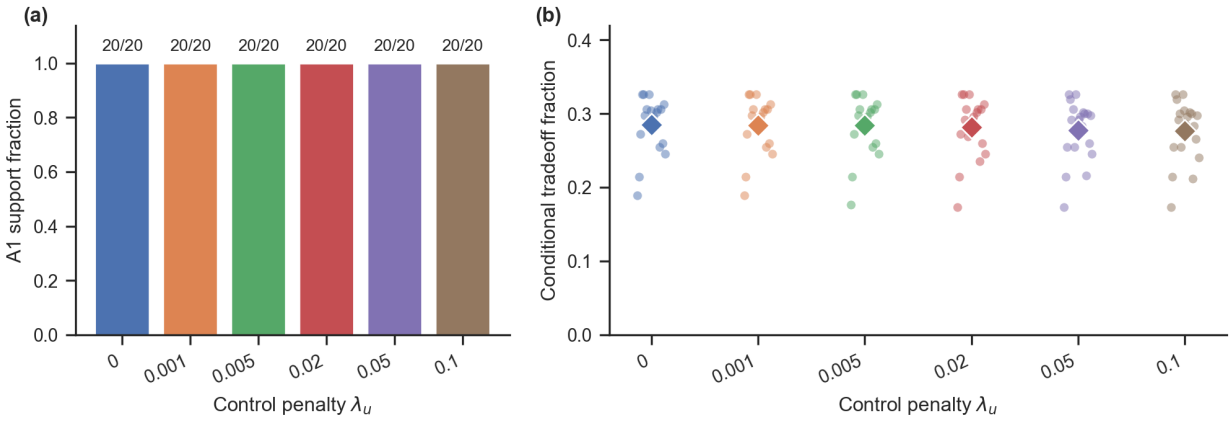

Figure 4: Targeted A1 short-vs-long horizon tradeoff sweep. Each condition uses 20 paired seeds. (a) All control-penalty settings support the predefined multi-horizon tradeoff criterion. (b) Seed-level conditional tradeoff fractions, defined as the fraction of short-horizon improvement intervals that also show long-horizon worsening or increased coupled spectral radius. Diamonds show condition means.

8 Result 4: Does the Phenomenon Generalize?

Generalization is partial (Figure 6). Across 90 completed architecture/task extension runs, C1 deployed-loss divergence is supported in 42/90 runs. The effect transfers cleanly to the tanh RNN, GRU, and low-rank

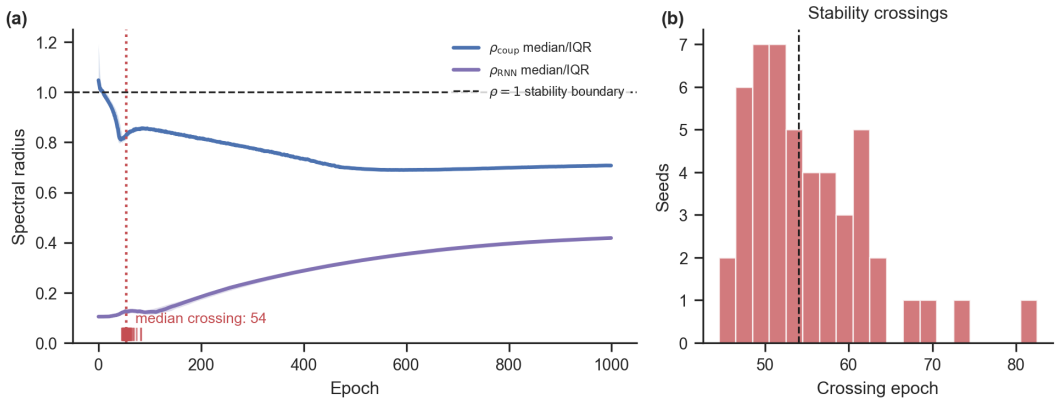

Figure 5: Coupled-system spectral analysis for the original setting. (a) Median coupled and RNN-only spectral radii across 50 seeds, with interquartile bands; the dashed line marks the $\rho = 1$ stability boundary and tick marks show per-seed crossing epochs. (b) Distribution of stability-crossing epochs. The coupled spectral radius crosses below the stability boundary early in training, whereas the RNN-only spectral radius remains far from the relevant stability threshold.

variants: each shows C1 support in 10/10 seeds. The tracking task shows partial support, with deployed-loss divergence in 8/10 seeds. The simple ring path-integration variant does not support C1 under the current task definition: 0/10 seeds show deployed-loss divergence and the mean deployed-loss gap is only 5.4×10^{-4}.

The negative ring result is interpretable as a boundary condition rather than a direct contradiction of the original double-integrator claim. The simple ring task gives the learner sin(error), cos(error), and velocity, with a fixed target. This makes the deployed state-action mapping comparatively easy for the open-loop student to imitate. We therefore ran a harder partial-observation moving-target ring follow-up. It still shows only weak C1 support: 3/20 GRU seeds and 1/20 tanh seeds pass the deployed-loss criterion, with mean deployed-loss gaps 0.0837 and 0.0085, respectively. These results sharpen A2: the closed/open divergence transfers across architecture changes on the double-integrator-style control problem, but it does not automatically transfer to path-integration variants.

C3 generalizes only where the coupled spectrum is implemented. It is supported in 10/10 tanh RNN runs and 10/10 low-rank runs, but not reported for GRU or ring runs because the current spectral code does not linearize gated nonlinear controllers. This is another reporting lesson: a generalization claim about stability mechanisms should distinguish performance transfer from diagnostic transfer.

9 Claim-Level Audit Matrix

Table 3 is the main summary of the paper. It separates original claims C1–C3 from audit questions A1–A2, and attaches each to evidence, robustness or generalization results, and an actionable lesson.

10 Actionable Lessons for Closed-Loop RNN Studies

The goal of the audit is to produce reusable reporting practices. We propose the following checklist for closed-loop RNN papers.

Report deployed closed-loop performance, not only open-loop or teacher-forced loss. The central risk is that a model can imitate a teacher under fixed inputs but fail when its own actions determine future inputs.

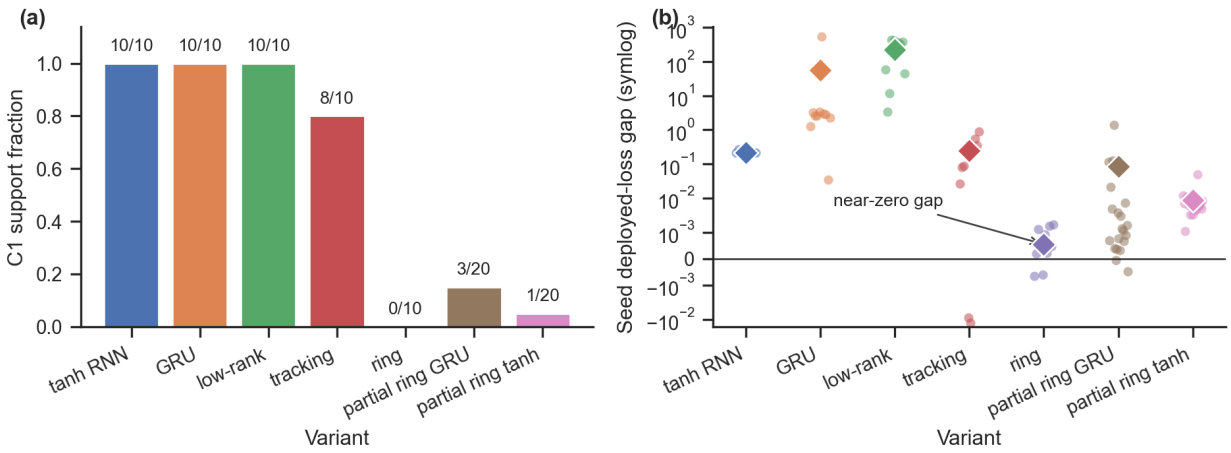

Figure 6: Generalization results across architecture and task variants. Standard variants use $n = 10$ paired seeds; the harder partial-observation ring variants use $n = 20$ paired seeds. (a) C1 support fractions with exact seed counts. (b) Seed-level deployed-loss gaps with mean diamonds on a symmetric-log scale, which permits tiny or negative gaps while still showing large architecture gaps. C1 transfers to tanh RNN, GRU, and low-rank variants, is partial on tracking, and is weak or absent in ring path-integration variants.

Table 3: Claim-level audit matrix for completed runs. C1–C3 are original claims under reproduction. A1–A2 are audit questions introduced here to test robustness, short/long horizon tradeoffs, and broader generalization.

Item	Evidence type	Original protocol	A1 audit	A2 audit	Lesson
C1: open/closed divergence	Paired seeds, effect size, CI, deployed loss	Reproduced: 50/50 seeds, loss gap 0.2186	Robust in most perturbations; absent under Adam and failed without clipping	42/90 extension runs; strong architecture transfer, weak path-integration transfer	Report deployed closed-loop loss, not only teacher-forced loss.
C2: stage structure	Stage detector, plateau duration, changepoints	Partial: slow-progress phase 50/50, strict three-stage 0/50	Slow phase robust; strict exit boundary fragile	Slow phase common; strict boundary varies by task	Stage claims require algorithmic criteria, not visual identification.
C3: coupled stability	Coupled spectral radius crossing	Reproduced: 50/50 seeds	Robust in all finite linearizable robustness settings	Holds for tanh/low-rank; not implemented for GRU/ring	Report coupled-system spectrum, not only RNN-only spectrum.
A1: protocol robustness and tradeoff	Perturbation grid and multi-horizon tradeoff metric	N/A	Mixed robustness plus targeted tradeoff support: 120/120, conditional fraction 0.281	N/A	Robustness claims need stress tests, explicit horizons, and interval-level criteria.
A2: broader generalization	Architecture and task extensions	N/A	N/A	Partial: 42/90 overall; architecture variants yes, path-integration weak	Closed-loop effects depend on task structure, environment coupling, and observability.

Use paired-seed comparisons when contrasting open-loop and closed-loop training. The clean comparison uses the same initialization, architecture, optimizer budget, and evaluation rollouts while changing only the training regime.

Define learning stages algorithmically. If a paper claims stages, it should specify transition criteria, minimum plateau duration, and how often each stage appears across seeds.

Analyze the coupled agent-environment system. RNN-only spectra can miss stability changes introduced by the environment. Coupled diagnostics should be reported whenever a mechanistic stability claim is made.

Report feedback strength and rollout horizon. These parameters determine how strongly actions affect future observations and can control whether the closed-loop phenomenon appears.

Report failure rates, not just average curves. Means can hide unstable runs, recovery failures, or seed-dependent plateaus.

Include at least one stress test. Optimizer, initialization, noise, feedback strength, and horizon perturbations are inexpensive compared with the interpretive cost of an untested mechanistic claim.

11 Discussion

This project treats Ger and Barak's framework as a case study in reproducibility for interactive recurrent systems. The completed audit supports the two strongest original claims in the original setting: closed-loop and open-loop training diverge under deployment, and the coupled agent-environment system undergoes a stability transition. It also supports A1 when the tradeoff is tested directly with short and long rollout horizons. The audit identifies two important limits. First, the stage claim depends strongly on how stages are defined. A slow-progress regime is reproducible, but a robust three-stage sequence with an objective second boundary is not recovered by our detectors. Second, generalization is conditional: the divergence transfers across several architectures and a tracking-control task, but is weak or absent in ring path-integration variants.

These mixed results are the reason a claim-level audit is useful. A figure-level reproduction would likely emphasize that the main closed-loop/open-loop loss gap appears. A methodological reproducibility study can say more: optimizer choice can remove the deployed-loss gap, missing gradient clipping can produce outright failure, path-integration-style variants can erase the gap even after adding partial observation and moving targets, spectra are only available for linearizable controllers unless local linearization is added, and stage claims should be accompanied by algorithmic boundaries rather than visual annotations alone. This is especially important for computational neuroscience, where RNNs are often used as mechanistic models and where closed-loop behavior is closer to biological learning than fixed-dataset supervision.

12 Limitations

The audit does not cover every RNN architecture, every control task, or full reinforcement-learning algorithms. The path-integration extensions are intentionally lightweight and should be interpreted as boundary-condition tests rather than a complete neuroscience benchmark. The A1 tradeoff metric is interval-level and diagnostic: it shows that short-horizon improvement often coincides with long-horizon or stability costs, but it does not by itself identify a unique causal mechanism. Finally, coupled spectral analysis is implemented for linearizable controllers and environments, but not yet for GRUs or other gated nonlinear controllers.

13 Artifact

The accompanying repository provides a config-driven implementation with smoke and full-run modes, raw CSV outputs, claim tables, and figure-generation scripts. The artifact is organized around the audit protocol rather than notebook-only reproduction: each experiment records the config, seed, device, metrics, and per-epoch time series. Full-run results can be regenerated with `bash scripts/run_full_audit.sh`; a lightweight validation run is available with `bash scripts/run_smoke.sh`. The targeted follow-up experiments for C2, A1, and the harder A2 path-integration setting can be launched with `bash scripts/run_targeted_c2_a1_a2.sh`.

References

Yoav Ger and Omri Barak. Learning dynamics of RNNs in closed-loop environments. In *Advances in Neural Information Processing Systems*, poster, 2025. OpenReview: `https://openreview.net/forum?id=P63bBMXCIH`.

Omri Barak. Recurrent neural networks as versatile tools of neuroscience research. *Current Opinion in Neurobiology*, 46:1–6, 2017.

Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems*, 2015.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

Niru Maheswaranathan, Alex H. Williams, Matthew D. Golub, Surya Ganguli, and David Sussillo. Universality and individuality in neural dynamics across large populations of recurrent networks. In *Advances in Neural Information Processing Systems*, 2019.

Valerio Mante, David Sussillo, Krishna V. Shenoy, and William T. Newsome. Context-dependent computation by recurrent dynamics in prefrontal cortex. *Nature*, 503:78–84, 2013.

Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d'Alché-Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research: a report from the NeurIPS 2019 reproducibility program. *Journal of Machine Learning Research*, 22(164):1–20, 2021.

Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 627–635, 2011.

David Sussillo and Omri Barak. Opening the black box: low-dimensional dynamics in high-dimensional recurrent neural networks. *Neural Computation*, 25(3):626–649, 2013.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.